

South China Normal University
Aberdeen Institute of Data Science and Artificial Intelligence

JC2001 Introduction to Software Engineering — Group Project

CampusFix: To Design and Implement a Campus Facility Reporting and Work-Order Management System

Project Proposal

Group number: Group 9

Programme of study: BSc (BMIS)

Date: 25 September 2026

Group Members

No.	Student ID	Name
1	20253803046	Xu Kunqin
2	20253803052	Lin Zhitao
3	20253803086	Jiang Yuhan
4	20253803060	Xiong Xiong
5	20253803056	Shu Xiyue
6	20253803024	Zhang Yue
7	20253803098	Wu Kaiyu
8	20253803006	Pan Yingyue
9	20253803018	Xiang Chen
10	20253803058	Peng Zhaobin

1. Problem

Campus facility problems are reported through scattered channels — corridor conversations, group chats and informal emails — and the descriptions that arrive are inconsistent. Location, category and urgency are frequently missing, so whoever receives a report has to go back and ask for the details that should have been captured at the start.

Reporters have no way to follow progress after submitting. They ask again and again, and each enquiry costs time on both sides. Administrators, meanwhile, cannot easily see which tickets are still waiting, which have been open too long, or which are likely duplicates of a report already in the system.

Technicians work without a central workbench. Work is handed out verbally, and the record of what was actually done, and when, is thin or missing altogether.

Above all, nothing is traceable. After the fact it is difficult to establish who reviewed a report, when it was assigned, who carried out the repair, and whether the reporter ever confirmed it. Without that record, recurring faults stay invisible and responsibility stays unclear.

2. Solution

We propose CampusFix, a web-based proof of concept for campus facility reporting and work-order management. A reporter submits a problem through a single form that captures category, location, description and photographs; a dispatch administrator reviews, classifies, prioritises and assigns it; a technician records the work carried out; and the reporter confirms the result or asks for rework.

The system is a modular monolith: a React and TypeScript front end, a REST API back end and a relational database, with private storage for attachments, started together through Docker Compose. Every status change is routed through a single workflow service that owns the state machine, so the database is never treated as a status field that clients may edit directly.

Tickets move through an explicit set of states — submitted, pending assignment, assigned, in progress, pending confirmation, closed, rejected and cancelled — and each transition is exposed as a business-meaningful action rather than a general

status update. Illegal transitions are rejected with a clear, user-facing error instead of being silently accepted.

Authorisation is enforced on the server for both role and resource relationship, so a technician cannot act on a ticket assigned to somebody else. Ticket rows carry a version number and status updates use optimistic locking, so two people acting at the same moment cannot silently overwrite each other's work.

Every status change, assignment change and comment is written to an append-only event log within the same database transaction as the update itself. Attachments are stored under random keys and served only through an authorised endpoint, and the dashboard aggregates live ticket data rather than displaying hard-coded figures.

3. Objectives

With this project, we intend to achieve the following:

- **(a)** To deliver, by 20 November 2026, a working PoC that completes the full ticket closed loop — submit, review, assign, process, confirm, close — demonstrated end to end.
- **(b)** To ensure that the three roles (Reporter, Technician, Admin) can only view and act on data within their authorisation, enforced on the server rather than in the interface.
- **(c)** To retain attachments, comments, status changes and responsibility changes as a complete, append-only timeline, so that any ticket can be audited afterwards.
- **(d)** To generate basic statistics from real ticket data — counts by status, category and building, and the elapsed time from creation to closure — so that administrators can identify backlogs and recurring problems.
- **(e)** To provide a reproducible installation, run, test and demonstration process that succeeds in a clean environment.
- **(f)** To document the requirements, modelling, design, implementation and testing in a technical report of 40–60 pages and a user manual of 8–12 pages, with a traceability matrix linking every requirement to its test.

4. Benefits

Reporters gain a single entry point that captures the necessary information up front, and can see the current status and who is responsible without having to ask.

Technicians receive one workbench listing the tickets assigned to them, together with a standard way to record what was done and attach the result. Administrators can see the backlog by status, category and building, and can identify duplicate reports filed for the same location.

For the university, the principal benefit is the audit trail: every review, assignment, repair and confirmation is recorded with its actor and timestamp. That turns recurring faults into visible patterns that can be acted on, instead of anecdotal complaints that are never aggregated.

For the team, the project is a genuine engineering exercise rather than a form over a database. It requires a state machine with transition authorisation, concurrent update control, transactional consistency between state and audit records, private file handling, and a delivery package that another person can rebuild from scratch. The PoC deliberately makes no claim to be a production service and uses no real student or staff data.

5. Timeline

Phase	Description	End date
Phase 1	Problem interviews, requirement list, use cases, prototype and this proposal; P0 scope frozen with the supervisor	25 September 2026
Phase 2	Data model, API contract and error format; authentication; ticket creation; report skeleton and first diagrams; workbenches	9 October 2026
Phase 3	Review, assignment, processing, confirmation and the event timeline; the complete closed loop demonstrable; first report chapters drafted	23 October 2026
Phase 4	Attachments, statistics, exception paths and automated tests; design and implementation chapters drafted; traceability matrix populated	6 November 2026
Phase 5	Report and user manual completed, presentation slides and recorded demonstration, reproducible delivery package	20 November 2026

Phase 1 completes at kick-off on 25 September 2026; the eight-week cycle then runs to 20 November 2026 as four two-week phases. Report writing runs in parallel from Phase 2.

6. Team and Responsibilities

Ten members work in the ten roles below, each named against the activities they own in Tables 3 to 6. Testing is not a separate team: every developer runs module tests on their own component, one back-end member owns the full-system tests and

the evidence for the report, and the front end runs the closed-loop integration tests and raises issues to the back end. The API contract is frozen in Phase 2 so the front end is built against a stable interface.

Member	Role	Responsibilities
Xu Kunqin	PM	Scope, schedule, supervisor liaison, project updates, workload profile; cross-phase integration and delivery coordination
Lin Zhitao	Backend — Workflow	Authentication, the ticket state machine, transition authorisation, optimistic locking, the event log and transactions; also owns the back-end core infrastructure and the full-system tests
Jiang Yuhan	Backend — Ticket & Location	Ticket data, location management, filtering and pagination, analytics endpoints and indexes; also owns the shared error format and co-owns the API contract
Xiong Xiong	Backend — Attachment & Test	Attachments, comments, containers, migrations, seed data and packaging; also owns the database layer and co-owns the API contract
Shu Xiyue	Frontend — Admin & Technician	Dispatch workbench, technician workbench, location management and analytics dashboard; also owns the shared component library and the mock layer
Zhang Yue	Frontend — Reporter	Login, report creation, my reports and ticket detail; also owns the front-end scaffold
Wu Kaiyu	Report — Design	Chapter 4 on design and architecture, plus the architecture, class and sequence diagrams
Pan Yingyue	Report — Requirements	Chapters 2 and 3, plus the use case, state machine, context and business process diagrams; leads the user manual
Xiang Chen	Report — Editor	Report structure, contents and figure lists, referencing and figure numbering; writes Chapters 1 and 7; leads the report group; consistency check and final PDF
Peng Zhaobin	Report — Implementation & Testing	Chapters 5 and 6 on implementation and testing; provides the installation, configuration and startup sections of the user manual

7. Action Plan

The activities required to achieve each objective are listed below, with the member responsible, the deadline and the current progress.

7.1 Objective 1 — To deliver the complete ticket closed loop with server-side authorisation

Action	Assigned to	Deadline	Progress
--------	-------------	----------	----------

Conduct problem interviews, write the proposal and freeze the P0 scope with the supervisor	Xu Kunqin	25 September 2026 (Phase 1)	In progress
Design the data model and draft the REST API contract, including the error format	Jiang Yuhan	9 October 2026 (Phase 2)	Not started
Freeze the REST API contract, control changes to it and keep front end and back end aligned	Xu Kunqin	9 October 2026 (Phase 2)	Not started
Implement login, logout, session handling and server-side role authorisation	Lin Zhitao	9 October 2026 (Phase 2)	Not started
Implement ticket creation, list, filtering and detail endpoints	Jiang Yuhan	9 October 2026 (Phase 2)	Not started
Build Docker Compose startup, database migrations and anonymised seed data	Xiong Xiong	9 October 2026 (Phase 2)	Not started
Implement the ticket state machine, transition authorisation and optimistic locking	Lin Zhitao	23 October 2026 (Phase 3)	Not started
Implement the review, assign, start, resolve, confirm and rework actions, with their mandatory-field rules	Lin Zhitao	23 October 2026 (Phase 3)	Not started
Build the reporter pages: create report, my reports and ticket detail	Shu Xiyue	23 October 2026 (Phase 3)	Not started
Build the dispatch workbench, including review, prioritisation and assignment	Zhang Yue	23 October 2026 (Phase 3)	Not started
Build the technician workbench and the action controls permitted by each state	Zhang Yue	23 October 2026 (Phase 3)	Not started
Connect the front end to the back end and run the closed-loop integration tests for the three roles	Shu Xiyue	23 October 2026 (Phase 3)	Not started

7.2 Objective 2 — To make every ticket traceable, consistent and measurable

Action	Assigned to	Deadline	Progress
Implement the append-only ticket event log with public and internal visibility	Lin Zhitao	23 October 2026 (Phase 3)	Not started
Implement the timeline view and public comments on the ticket detail	Shu Xiyue	6 November 2026 (Phase 3)	Not started

page		4)	
Implement image upload with type and size validation, and authorised private download	Xiong Xiong	6 November 2026 (Phase 4)	Not started
Implement the analytics endpoints: counts by status, category and building, and time to closure	Jiang Yuhan	6 November 2026 (Phase 4)	Not started
Build the analytics dashboard reading live ticket data	Zhang Yue	6 November 2026 (Phase 4)	Not started
Add indexes for the common filter fields and verify pagination behaviour	Jiang Yuhan	6 November 2026 (Phase 4)	Not started
Record the key technical and design decisions for the design chapter	Xu Kunqin	6 November 2026 (Phase 4)	Not started
Verify that a status change and its event record commit in one transaction under concurrency	Lin Zhitao	6 November 2026 (Phase 4)	Not started
Raise an issue against the back end for every integration failure and retest after the fix	Zhang Yue	20 November 2026 (Phase 5)	Not started

7.3 Objective 3 — To document the project in the technical report

Action	Assigned to	Deadline	Progress
Set up the report skeleton: contents, figure and table lists, chapter template and referencing style	Wu Kaiyu	9 October 2026 (Phase 2)	Not started
Produce the context model, the business process model and the architecture diagram	Xiang Chen	9 October 2026 (Phase 2)	Not started
Write Chapter 1, Introduction and Project Context	Wu Kaiyu	23 October 2026 (Phase 3)	Not started
Write Chapter 2, Functional and Non-functional Requirements, with traced requirement IDs	Pan Yingyue	23 October 2026 (Phase 3)	Not started
Produce the use case diagram and the UC-01 to UC-05 descriptions, with success and exception paths	Pan Yingyue	23 October 2026 (Phase 3)	Not started
Write Chapter 4, System Design and Architecture, and produce the class and sequence diagrams	Xiang Chen	6 November 2026 (Phase 4)	Not started

Collect material from the developers and write Chapter 5, Implementation	Peng Zhaobin	6 November 2026 (Phase 4)	Not started
Write Chapter 6, Testing and Evaluation, with results and evaluation against each requirement	Peng Zhaobin	6 November 2026 (Phase 4)	Not started
Write Chapter 7, Conclusions and Future Work, and compile the References list	Wu Kaiyu	20 November 2026 (Phase 5)	Not started
Maintain the traceability matrix from requirement to test result	Wu Kaiyu	20 November 2026 (Phase 5)	Not started
Proof-read the report for consistency between problem, requirements, design, implementation and testing	Wu Kaiyu	20 November 2026 (Phase 5)	Not started
Format the report to the required layout and submit it as a PDF	Wu Kaiyu	20 November 2026 (Phase 5)	Not started

7.4 Objective 4 — To deliver a reproducible, tested and presented PoC

Action	Assigned to	Deadline	Progress
Write and maintain the module and API integration tests, including the full permission matrix	Lin Zhitao	6 November 2026 (Phase 4)	Not started
Write and run the end-to-end test for the three-role closed loop, including exception paths	Lin Zhitao	6 November 2026 (Phase 4)	Not started
Run the performance test against 500 seed tickets and verify each non-functional requirement	Lin Zhitao	6 November 2026 (Phase 4)	Not started
Write the 8–12 page user manual and verify it by executing it in a clean environment	Peng Zhaobin	20 November 2026 (Phase 5)	Not started
Package the PoC archive and confirm reproduction on a clean machine	Xiong Xiong	20 November 2026 (Phase 5)	Not started
Prepare the presentation slides and record the 15-minute MP4 with all members presenting	Xiang Chen	20 November 2026 (Phase 5)	Not started
Submit the project updates and circulate the workload profile for signature by all members	Xu Kunqin	20 November 2026 (Phase 5)	Not started